

Лабораторная работа №4

Вычисление наибольшего общего делителя

Кюнкриков Д.С.

Российский университет дружбы народов (РУДН), Москва, Россия

- Цель и задачи
- Теория: НОД и алгоритмы Евклида
- Реализация (Python)
- Примеры работы
- Выводы

Цель: реализовать алгоритмы вычисления НОД и (для расширенных версий) коэффициенты Безу.

Задачи: - Евклид - Бинарный Евклид (Стейн) - Расширенный Евклид (Безу) -
Расширенный бинарный Евклид

- Лемма Евклида: если $a = qb + r$, то $\gcd(a, b) = \gcd(b, r)$.
- Тождество Безу: существуют x, y , что $ax + by = \gcd(a, b)$.

Итеративно выполняем: - $r \leftarrow a \bmod b$ - $(a, b) \leftarrow (b, r)$
пока $b \neq 0$. Ответ: последний ненулевой остаток.

Используем свойства: - если оба чётные: выносим множитель 2; - делим чётное на 2;
- для нечётных используем вычитание $|a - b|$.

- **Расширенный Евклид** ведёт коэффициенты x, y вместе с остатками.
- **Расширенный бинарный** поддерживает инварианты:
 - $u = Aa + Bb$
 - $v = Ca + Db$

Реализация: Алгоритм Евклида (Python)

```
def euclidean_gcd(a,b):  
    r_prev, r_curr = a,b  
  
    while r_curr !=0:  
        r_next = r_prev % r_curr  
        r_prev = r_curr  
        r_curr = r_next  
  
    d = r_prev  
    return d  
  
vala = 12345  
valb = [24690, 54321, 12541]  
for val in valb:  
    print(f"GCD({vala}, {val}) = {euclidean_gcd(vala, val)}")
```

Рисунок 1: Программная реализация алгоритма Евклида

Реализация: Бинарный алгоритм Евклида (Python)

```
#2 Бинарный алгоритм Евклида
def binareuc_gcd(a,b):
    g = 1

    while a%2 == 0 and b%2 ==0:
        a //=2
        b //= 2
        g *= 2

    u, v = a, b

    while u != 0:
        while u% 2 ==0:
            u //= 2
        while v% 2 ==0:
            v //= 2
        if u >= v:
            u = u-v
        else:
            v = v-u

    d = g * v

    return d

vala = 12345
valb = [24690, 54321, 12541]
for val in valb:
    print(f"GCD({vala}, {val}) = {binareuc_gcd(vala, val)}")
```

Рисунок 2: Программная реализация бинарного алгоритма Евклида

Реализация: Расширенный алгоритм Евклида

Возвращаем (d, x, y) , где $ax + by = d$.

```
def extendedeuc_gcd(a,b):
    r_prev, r_curr = a,b
    x_prev, x_curr = 1, 0
    y_prev, y_curr = 0, 1

    while r_curr !=0:
        q = r_prev // r_curr
        r_next = r_prev % r_curr
        x_next = x_prev - q * x_curr
        y_next = y_prev - q * y_curr

        r_prev, r_curr = r_curr, r_next
        x_prev, x_curr = x_curr, x_next
        y_prev, y_curr = y_curr, y_next

    d, x, y = r_prev, x_prev, y_prev
    return d, x, y_curr

d1, x1, y1 = extendedeuc_gcd(105,91)
d2, x2, y2 = extendedeuc_gcd(154, d1)
print(f"GDC(105,91) = {d1, x1, y1}")
print(f"final GDC(154,7) = {d2, x2, y2}")
```

Рисунок 3: Программная реализация расширенного алгоритма Евклида

Реализация: Расширенный бинарный алгоритм Евклида

```
def extendbineuc_gcd(a,b):
    g = 1
    while a%2 == 0 and b%2 == 0:
        a //= 2
        b //= 2
        g *= 2

    u, v = a, b
    A, B = 1, 0
    C, D = 0, 1

    while u != 0:
        while u % 2 == 0:
            u //= 2
            if A % 2 == 0 and B % 2 == 0:
                A //= 2
                B //= 2
            else:
                A = (A + b) // 2
                B = (B - a) // 2
            print("v: " + str(v))
```

Рисунок 4: Программная реализация расширенного бинарного алгоритма Евклида

```
while v%2==0:
    v //= 2
    if C % 2 == 0 and D % 2 == 0:
        C //= 2
        D //= 2
    else:
        C = (C+b) // 2
        D = (D-a) // 2
    print("u: " + str(u))

    if u >= v:
        u = u - v
        A = A - C
        B = B - D
    else:
        v = v - u
        C = C - A
        D = D - B

    d = g * v
    x = C
    y = D
    return d, x, y
```

```
dbin,xbin,ybin = extendbineuc_gcd(105, 91)
print(f"GCD(105,91) = {dbin,xbin,ybin}")
```

Рисунок 5

Результат работы программы

Для пар $(12345, 24690)$, $(12345, 54321)$, $(12345, 12541)$ значения НОД у всех реализаций должны совпадать с `math.gcd`.

```
GCD(12345, 24690) = 12345
GCD(12345, 54321) = 3
GCD(12345, 12541) = 1
GCD(12345, 24690) = 12345
GCD(12345, 54321) = 3
GCD(12345, 12541) = 1
GDC(105, 91) = (7, -6, -15)
final GDC(154, 7) = (7, 0, -22)
v: 91
u: 7
u: 7
u: 7
GDC(105, 91) = (7, -6, 7)
```

Для $(105, 91)$ проверяем равенство $105x + 91y = d$.

- Реализованы 4 алгоритма вычисления НОД.
- Для расширенных алгоритмов получены коэффициенты Безу.
- Корректность проверена сравнением с `math.gcd` и равенством $ax + by = d$.